



D.Y. Patil Academic Education Excellence Federation's

Dr. D. Y. Patil Technical Campus

Polytechnic | Engineering | MCA

(Approved by AICTE, Recognized by DTE, Govt. of Maharashtra, Affiliated to Savitribai Phule Pune University, Pune)

Sr.No.32/1/A/7, Near Talegaon Railway Station, A/P Varale, Talegaon Dabhade, Tal-Maval, Dist-Pune 410 507

Website: www.dypatiltcs.com Email: principal@dypatiltcs.com

AICTE ID: 1- 43358498993 | SPPU PUN CODE: CMMP021930 | ISHE CODE: C -71504 | DTE CODE: 6991

Dr. D. Y. Patil
Founder Dr. D. Y. Patil Group

Dr. Sushant V. Patil
Chairman

Adv. Anuja Sushant Patil
Director

Dr. Suresh Shirbahadurkar
Principal

DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

TOPIC: Data Structure Using C++

Prepared By: Vedshree Aswale

❖ Data Structure:

Definition:

- Data is a collection of numbers, alphabets, and special symbols used to represent information.
- Data structure is the proper way of organizing and arranging data in a computer.
- The systematic representation of data in the main memory is called a data structure.
- Data can be organized and stored in different ways depending on the requirement.
- The logical or mathematical model used to organize a particular type of data is called a data structure.
- In C++, a data structure is a way of storing and organizing data so that it can be accessed, processed, and used efficiently.

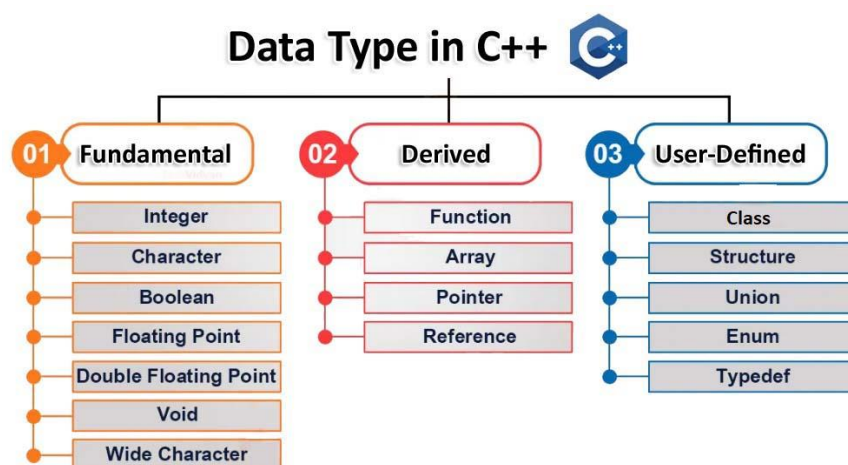
Need of Data Structure:

- Data structures are an important way of organizing and managing data in a computer.
- They provide different ways of storing and organizing data in C++.
- They help us understand the relationship between one data element and another.
- They help to store data in a logical and systematic manner.
- Data structures allow data to grow and shrink dynamically when required.
- When data is organized properly, it can be accessed and processed efficiently using suitable algorithms.
- Data structures are used with efficient algorithms, which makes it possible to manage and process large amounts of data efficiently.

❖ Abstract Data Type(ADT)

- Abstract Data Type (ADT) is a logical model that defines a type of data along with a set of values and a set of operations that can be performed on those values.
- The definition of an ADT specifies what operations are to be performed on the data without giving detailed information about how these operations will be implemented.
- It does not specify how the data will be organized in memory or which algorithms and data structures will be used to implement the operations.
- It is called “Abstract” because it shows only the essential details and hides the unnecessary implementation details.
- In C++, ADTs can be implemented using classes, structures, and functions, while the internal implementation can be hidden using encapsulation and access specifiers such as private and public.
- The process of providing only the essential details and hiding the unwanted implementation details is known as abstraction.
- For example, in C++, we use data structures such as stack, queue, linked list, tree, and graph to store and manage data and perform various operations. However, the user does not need to know all the internal implementation details of these operations.
- It means the user understands what the data structure does and which operations can be performed on it, but does not need to know how the data is organized in memory or how the operations are implemented internally.
- For example, in a Stack, the user can perform operations such as push(), pop(), and peek(). The user only needs to know the purpose of these operations, not the internal code or memory management used to implement them.
- ADT can be considered as a black box that hides the internal structure and implementation of the data type while providing only the required operations to the user.

❖ Classification of data structure:



❖ Operations on Data Structures in C++

The basic operations that are performed on data structures in C++ are as follows:

1. Insertion:

It is the operation of **adding a new data element** to a data structure.

2. Deletion:

It is the operation of **removing an existing data element** from a data structure.

3. Searching:

It is the operation of **finding the location of a particular data element** within a given data structure.

4. Sorting:

It is the operation of **arranging data elements in a logical order**, such as **ascending or descending order**

5. Traversing:

It is the operation of **accessing each and every element** of a data structure one by one.

6. Merging:

It is the operation used to **combine the data elements of two data structures into a single data structure**.

❖ Algorithms Complexity:

1. Time complexity:-

- Time complexity of a program or algorithm is the amount of computer time required for the program or algorithm to execute until completion.
- The amount of time required for the execution of a given C++ program or algorithm is known as its time complexity.
- While calculating time complexity, we develop a frequency count for the key statements or basic instructions that are important in the algorithm.
- Frequency count represents how many times a particular statement is executed or visited by the program control during execution.
- Time complexity is generally represented using Big-O notation, such as $O(1)$, $O(n)$, $O(n^2)$, etc
- Example:

Sr. No	Statement	Frequency Count
1	sum = 0;	1
2	for(i = 1; i <= n; i++)	n+1
3	sum = sum + i;	n
4	cout << sum;	1

So the time complexity is $2n+3$ and if we present this time complexity using Big-Oh notation then it will be $O(n)$.

2.Space complexity:-

- space complexity of a program or algorithm is the amount of memory required by the program or algorithm to execute and complete its operation.
- The amount of memory required for the execution of a given C++ program or data structure algorithm is known as its space complexity.
- Space complexity is important when implementing data structures because different data structures may require different amounts of memory.
- The space complexity consists of two parts:
 1. Fixed space part: It includes space for instructions, for simple variables, fixed size structured variables and constants.
 2. Variable space part: It consists of space needed by structured variables whose size depends on particular value of variables.

Space Complexity = Fixed part space + Variable part space

❖ Searching Techniques:

- 1.Linear Search
- 2.Binary Search

1.Linear Search

- Linear Search is a searching algorithm in which the key element is searched sequentially in a given data structure.
- It is also called Sequential Search.
- The key element is compared with each element one by one.
- Searching starts from the first element and continues until the key is found or the end of the data structure is reached.
- It is suitable for small numbers of elements but is inefficient for large datasets.
- Time Complexity: $O(n)$

Algorithm for Linear Search in C++

1. Start
2. Read n (number of elements).
3. Read n array elements into A.
4. Read the key element to be searched.
5. Set $i = 0$ and $flag = 0$.
6. Compare $A[i]$ with key.
 - If $A[i] == key$, set $flag = 1$ and go to Step 8.
 - Otherwise, increment $i = i + 1$.
7. If $i < n$, go to Step 6.
8. If $flag == 1$, print "Element is found"; otherwise, print "Element is not found".
9. Stop

Write a C++ Program to implement Linear search.

```
#include <iostream>

using namespace std;

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int key;

    cout << "Enter element to search: ";
    cin >> key;
```

```

for(int i = 0; i < 5; i++)
{
    if(a[i] == key)
    {
        cout << "Element found";
        return 0;
    }
}

cout << "Element not found";
return 0;
}

```

2.Binary Search

- Binary search works only on a sorted array.
- Set lower = 0 and upper = n - 1.
- Calculate the middle index: $\text{mid} = (\text{lower} + \text{upper}) / 2$.
- Compare arr[mid] with the search key:
- If $\text{arr}[\text{mid}] == \text{key} \rightarrow$ element found.
- If $\text{key} < \text{arr}[\text{mid}] \rightarrow$ search in the left half ($\text{upper} = \text{mid} - 1$).
- If $\text{key} > \text{arr}[\text{mid}] \rightarrow$ search in the right half ($\text{lower} = \text{mid} + 1$).
- Repeat until the element is found or $\text{lower} > \text{upper}$.
- Time Complexity: $O(\log n)$.

Algorithm for Binary Search in C++

1. Start
2. Read the number of elements n.
3. Read the elements of the sorted array A.
4. Read the element to be searched as key.
5. Set low = 0 and high = n - 1.
6. Repeat while low <= high:
 - Calculate $\text{mid} = (\text{low} + \text{high}) / 2$.
 - If $\text{A}[\text{mid}] == \text{key}$, print "Element found" and stop.
 - If $\text{key} < \text{A}[\text{mid}]$, set high = mid - 1.
 - Otherwise, set low = mid + 1.
7. If low > high, print "Element not found".
8. Stop

Write a C++ Program to implement Binary search.

```
#include <iostream>

using namespace std;

int main()
{
    int a[5] = {10, 20, 30, 40, 50};
    int key, low = 0, high = 4, mid;
    cout << "Enter element to search: ";
    cin >> key;
    while(low <= high)
    {
        mid = (low + high) / 2;
        if(a[mid] == key)
        {
            cout << "Element found";
            return 0;
        }
        else if(key < a[mid])
            high = mid - 1;
        else
            low = mid + 1;
    }
}
```

```

cout << "Element not found";

return 0;

}

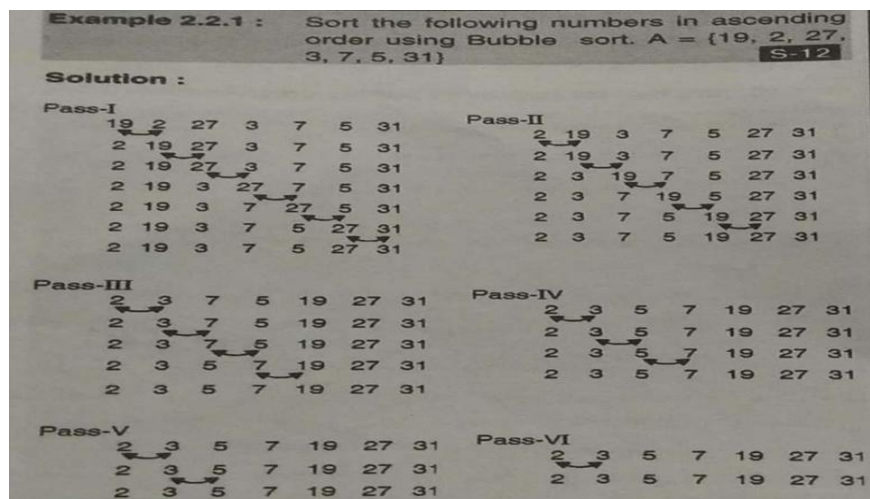
```

❖ Sorting Techniques:

1. Bubble Sort
2. Selection Sort
3. Insertion Sort
4. Radix Sort
5. Quick Sort
6. Merge Sort

1. Bubble Sort

- Bubble Sort is a simple sorting algorithm.
- It compares **two adjacent elements**.
- If the first element is greater than the second, they are swapped.
- After every pass, the largest element moves to the end.
- The process is repeated for the remaining unsorted elements.
- It is easy to understand and implement.
- It works directly on the original array.
- Therefore, it is an **in-place sorting algorithm**.
- Best-case time complexity is **$O(n)$** with an early-stop check.
- Average and worst-case time complexity is **$O(n^2)$** .



Algorithm for Bubble Sort in C++

1. Start with an array.
2. Set n as the number of elements.
3. Start the first pass.
4. Compare adjacent elements.
5. If the first element is greater, swap them.
6. Continue until the end of the unsorted part.
7. The largest element reaches the last position.
8. Repeat the process for the remaining elements.
9. Stop if no swapping occurs.
10. Print the sorted array.

Write a C++ Program to implement Bubble Sort.

```
#include <iostream>

using namespace std;

int main()
{
    int arr[] = {19,2,27,3,7,5,31};
    int n = 7;

    for (int i = 0; i < n - 1; i++)
    {
        for (int j = 0; j < n - 1 - i; j++)
        {
            if (arr[j] > arr[j + 1])
            {
                swap(arr[j], arr[j + 1]);
            }
        }
    }
}
```

```

cout << "Sorted Array: ";

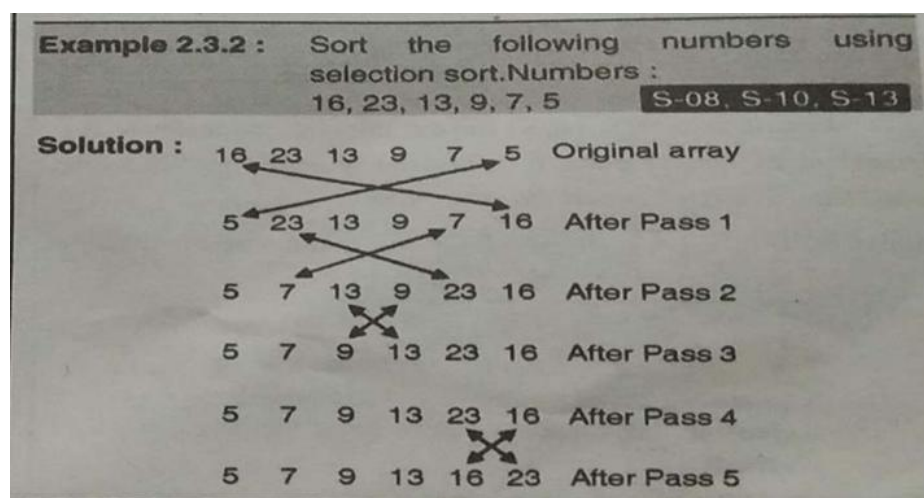
for (int i = 0; i < n; i++)
    cout << arr[i] << " ";

return 0;
}

```

2.Selection Sort

- Selection Sort is a comparison-based sorting algorithm.
- It divides the array into **sorted and unsorted parts**.
- Initially, the entire array is unsorted.
- It finds the smallest element from the unsorted part.
- The smallest element is placed at the beginning.
- The sorted part increases after every iteration.
- The process continues until the complete array is sorted.
- It requires fewer swaps than Bubble Sort.
- It is an **in-place sorting algorithm**.
- Best, average, and worst-case time complexity is $O(n^2)$



Algorithm for Selection Sort in C++

- Start with an unsorted array.
- Select the first position.
- Assume it contains the smallest element.
- Compare it with the remaining elements.
- Find the actual smallest element.
- Swap it with the first unsorted element.
- Move to the next position.
- Repeat the same process.
- Continue until the array is sorted.
- Print the sorted array

Write a C++ Program to implement Selection Sort.

```
#include <iostream>

using namespace std;

int main()
{
    int arr[] = {5, 3, 8, 4, 2};
    int n = 5;
    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }
    }
}
```

```

arr[j + 1] = key;
    }
    cout << "Sorted Array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

3. Insertion Sort

- Insertion Sort is a simple comparison-based algorithm.
- It works similar to arranging **playing cards in your hand**.
- It divides the array into sorted and unsorted parts.
- The first element is considered sorted.
- The next element is selected as the **key**.
- The key is compared with previous elements.
- Larger elements are shifted to the right.
- The key is inserted into its correct position.
- It is an **in-place sorting algorithm**.
- Best-case complexity is **O(n)** and average/worst-case complexity is **O(n²)**.

Algorithm for Insertion Sort in C++

1. Start with an array.
2. Consider the first element as sorted.
3. Select the second element as the key.
4. Compare the key with previous elements.
5. Shift larger elements one position to the right.
6. Insert the key in the correct position.
7. Select the next element.
8. Repeat the same process.
9. Continue until the last element.
10. Print the sorted array

Write a C++ Program to implement Insertion Sort.

```
#include <iostream>

using namespace std;

int main()
{
    int arr[] = {5, 3, 8, 4, 2};
    int n = 5;

    for (int i = 1; i < n; i++)
    {
        int key = arr[i];
        int j = i - 1;

        while (j >= 0 && arr[j] > key)
        {
            arr[j + 1] = arr[j];
            j--;
        }

        arr[j + 1] = key;
    }

    cout << "Sorted Array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}
```

4.Radix Sort

- **Radix Sort** is a non-comparison-based sorting algorithm.
- It is mainly used to sort **integers or numbers**.
- It sorts numbers digit by digit.
- It starts sorting from the **least significant digit (LSD)**, usually the units digit.
- Then it sorts according to the tens digit, hundreds digit, and so on.
- It uses a **stable sorting technique**, commonly Counting Sort, for each digit.
- The number of passes depends on the number of digits in the largest number.
- Radix Sort does not compare two numbers directly like Bubble Sort or Selection Sort.
- Its time complexity is **$O(d \times (n + k))$** , where d is the number of digits, n is the number of elements, and k is the range of each digit (normally 10).
- It is useful when there are many integers with a limited number of digits.

Algorithm for Radix Sort in C++

1. Find the largest element in the array.
2. Set place = 1 to represent the units digit.
3. Repeat while largest / place > 0.
4. Perform Counting Sort according to the digit at the current place.
5. Calculate the digit using:
 $\text{digit} = (A[i] / \text{place}) \% 10$
6. Count the occurrence of each digit from 0 to 9.
7. Find the cumulative count of each digit.
8. Place the elements into the output array according to their digits.
9. Copy the output array back to the original array.
10. Multiply place by 10 and repeat until all digits are processed.
11. Print the sorted array.

- a) Describe radix sort algorithm. Sort the following numbers in ascending order using radix sort.
12, 8, 25, 4, 66, 2, 98, 225.

Sorting of Given Numbers: -
Pass 1:

Bucket Elements	0	1	2	3	4	5	6	7	8	9
12			12							
8									8	
25						25				
4					4					
66							66			
2			2							
98									98	
225						225				

Output of Pass 1: 12 02, 04, 25, 225, 66, 8, 98

Pass 2:

Bucket Elements	0	1	2	3	4	5	6	7	8	9
12		12								
2	2									
4	4									
25			25							
225			225							
66							66			
8	8									
98										98

Output of Pass 2: 02, 04, 08, 12, 25, 225, 66, 98

Pass 3:

Bucket Elements	0	1	2	3	4	5	6	7	8	9
2	2									
4	4									
8	8									
12	12									
25	25									
225			225							
66	66									
98	98									

Sorted list: 2, 4, 8, 12, 25, 66, 98, 225

Write a C++ Program to implement Radix Sort.

```
#include <iostream>
using namespace std;

int getMax(int a[], int n)
{
    int max = a[0];

    for (int i = 1; i < n; i++)
    {
        if (a[i] > max)
            max = a[i];
    }

    return max;
}

void countingSort(int a[], int n, int place)
{

```

```

int output[100];

int count[10] = {0};

// Count digits
for (int i = 0; i < n; i++)
{
    int digit = (a[i] / place) % 10;
    count[digit]++;
}

// Cumulative count
for (int i = 1; i < 10; i++)
{
    count[i] = count[i] + count[i - 1];
}

// Place elements in output array
for (int i = n - 1; i >= 0; i--)
{
    int digit = (a[i] / place) % 10;

    output[count[digit] - 1] = a[i];
    count[digit]--;
}

// Copy output to original array
for (int i = 0; i < n; i++)
{
    a[i] = output[i];
}
}

void radixSort(int a[], int n)
{
    int max = getMax(a, n);

    // Sort according to each digit
    for (int place = 1; max / place > 0; place *= 10)
    {
        countingSort(a, n, place);
    }
}

int main()
{

```

```

int a[] = {170, 45, 75, 90, 802, 24, 2, 66};

int n = 8;

radixSort(a, n);

cout << "Sorted Array: ";

for (int i = 0; i < n; i++)
{
    cout << a[i] << " ";
}
return 0;
}

```

5. Merge Sort

- Merge Sort is a **divide-and-conquer** algorithm.
- It divides the array into two halves.
- Each half is divided again.
- This continues until single elements remain.
- A single element is considered sorted.
- The sorted elements are then merged.
- During merging, smaller elements are selected first.
- The two halves are combined into one sorted array.
- It has time complexity **$O(n \log n)$** in best, average, and worst cases.
- It requires additional memory for merging.

Algorithm for Merge Sort in C++

1. Start with an array.
2. Find the middle position.
3. Divide the array into two halves.
4. Apply Merge Sort to the left half.
5. Apply Merge Sort to the right half.
6. Compare elements of both halves.
7. Select the smaller element.
8. Continue until all elements are merged.
9. Copy the remaining elements.
10. Print the sorted array.

Write a C++ Program to implement Merge Sort.

```
#include <iostream>
using namespace std;

void merge(int arr[], int left, int mid, int right)
{
    int i = left;
    int j = mid + 1;
    int k = 0;

    int temp[100];

    while (i <= mid && j <= right)
    {
        if (arr[i] < arr[j])
        {
            temp[k++] = arr[i++];
        }
        else
        {
            temp[k++] = arr[j++];
        }
    }

    while (i <= mid)
        temp[k++] = arr[i++];

    while (j <= right)
        temp[k++] = arr[j++];

    for (i = left, k = 0; i <= right; i++, k++)
        arr[i] = temp[k];
}

void mergeSort(int arr[], int left, int right)
{
    if (left < right)
    {
        int mid = (left + right) / 2;

        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
    }
}
```

```

        merge(arr, left, mid, right);
    }
}

int main()
{
    int arr[] = {5, 3, 8, 4, 2};
    int n = 5;

    mergeSort(arr, 0, n - 1);

    cout << "Sorted Array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

6. Quick Sort

- Quick Sort is a **divide-and-conquer** algorithm.
- It selects an element called the **pivot**.
- The pivot is used to divide the array.
- Smaller elements are placed before the pivot.
- Larger elements are placed after the pivot.
- This process is called **partitioning**.
- The pivot gets its correct position.
- Quick Sort is then applied to the left and right parts.
- Average-case time complexity is **$O(n \log n)$** .
- Worst-case time complexity is **$O(n^2)$** when partitions are highly unbalanced

Algorithm for Quick Sort in C++

- Quick Sort is a **divide-and-conquer** algorithm.
- It selects an element called the **pivot**.
- The pivot is used to divide the array.
- Smaller elements are placed before the pivot.
- Larger elements are placed after the pivot.
- This process is called **partitioning**.
- The pivot gets its correct position.

- Quick Sort is then applied to the left and right parts.
- Average-case time complexity is **$O(n \log n)$** .
- Worst-case time complexity is **$O(n^2)$** when partitions are highly unbalanced

Write a C++ Program to implement Quick Sort.

```
#include <iostream>

using namespace std;

int partition(int arr[], int low, int high)
{
    int pivot = arr[high];
    int i = low - 1;

    for (int j = low; j < high; j++)
    {
        if (arr[j] < pivot)
        {
            i++;
            swap(arr[i], arr[j]);
        }
    }

    swap(arr[i + 1], arr[high]);

    return i + 1;
}

void quickSort(int arr[], int low, int high)
{
    if (low < high)
    {
```

```

        int p = partition(arr, low, high);

        quickSort(arr, low, p - 1);
        quickSort(arr, p + 1, high);
    }
}

int main()
{
    int arr[] = {5, 3, 8, 4, 2};
    int n = 5;

    quickSort(arr, 0, n - 1);

    cout << "Sorted Array: ";

    for (int i = 0; i < n; i++)
        cout << arr[i] << " ";

    return 0;
}

```

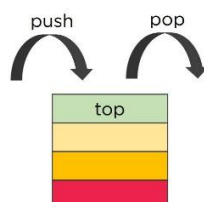
❖ Stack

A Stack is a linear data structure in which insertion and deletion of elements are performed from the same end, called the TOP.

Stack follows the LIFO principle:

LIFO = Last In, First Out

Stack



Example

If we insert:

10 → 20 → 30

The stack looks like:

TOP

↓

| 30 |

| 20 |

| 10 |

| |

If we perform **POP**, 30 will be removed first.

Operations of Stack

1. Push

Push is used to insert an element into the stack.

Example:

Push(10)

Push(20)

Push(30)

Stack:

| 30 | ← TOP

| 20 |

| 10 |

| |

2. Pop

Pop is used to remove the top element from the stack.

Example:

Pop()

30 will be removed.

3. Peek / Top

Peek returns the top element without removing it.

Example:

Peek()

Output:

30

4. IsEmpty

Checks whether the stack is empty.

If stack is empty → TRUE

Otherwise → FALSE

5. IsFull

Checks whether the stack is full when an array is used to implement the stack.

Write a C++ Program to implement Stack Operation.

```
#include <iostream>
```

```
using namespace std;
```

```
#define MAX 5
```

```
int stackArr[MAX];

int top = -1;


// Push operation
void push(int value)
{
    if (top == MAX - 1)
    {
        cout << "Stack Overflow\n";
    }
    else
    {
        top++;
        stackArr[top] = value;
        cout << value << " inserted into stack\n";
    }
}


// Pop operation
void pop()
{
    if (top == -1)
    {
        cout << "Stack Underflow\n";
    }
}
```

```
else
{
    cout << stackArr[top] << " deleted from stack\n";
    top--;
}
}

// Peek operation
void peek()
{
    if (top == -1)
    {
        cout << "Stack is empty\n";
    }
    else
    {
        cout << "Top element = " << stackArr[top] << endl;
    }
}

// Display operation
void display()
{
    if (top == -1)
    {
```

```
        cout << "Stack is empty\n";  
    }  
    else  
    {  
        cout << "Stack elements:\n";  
  
        for (int i = top; i >= 0; i--)  
        {  
            cout << stackArr[i] << endl;  
        }  
    }  
}
```

```
int main()  
{  
    push(10);  
    push(20);  
    push(30);  
  
    display();  
  
    peek();  
  
    pop();  
}
```

```

display();

return 0;

}

```

❖ Queue

A Queue is a linear data structure in which insertion is performed at one end called REAR, and deletion is performed at another end called FRONT.

Queue follows the FIFO principle:

FIFO = First In, First Out



Queue Data Structure

Example

If we insert:

10 → 20 → 30

The queue looks like:

FRONT		REAR
↓		↓
10	20	30

If we perform DEQUEUE, 10 will be removed first.

Operations of Queue

1. Enqueue

Enqueue is used to insert an element into the queue.

Insertion takes place at the REAR.

Example:

Enqueue(10)

Enqueue(20)

Enqueue(30)

Queue:

FRONT				REAR		
↓				↓		
	10		20		30	

2. Dequeue

Dequeue is used to remove an element from the queue.

Deletion takes place from the FRONT.

Example:

Dequeue()

10 will be removed.

3. Peek / Front

Returns the first element of the queue without deleting it.

Example:

Peek()

Output:

10

4. IsEmpty

Checks whether the queue contains no elements.

5. IsFull

Checks whether the queue is full when an array is used.

Write a C++ Program to implement Queue Operation.

```
#include <iostream>
using namespace std;

#define MAX 5

int queueArr[MAX];
int front = -1;
int rear = -1;

// Enqueue operation
void enqueue(int value)
{
    if (rear == MAX - 1)
    {
        cout << "Queue Overflow\n";
    }
    else
    {
        if (front == -1)
            front = 0;

        rear++;
        queueArr[rear] = value;

        cout << value << " inserted into queue\n";
    }
}

// Dequeue operation
void dequeue()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue Underflow\n";
    }
    else
    {
        cout << queueArr[front] << " deleted from queue\n";
    }
}
```

```

        front++;
    }
}

// Peek operation
void peek()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty\n";
    }
    else
    {
        cout << "Front element = " << queueArr[front] << endl;
    }
}

// Display operation
void display()
{
    if (front == -1 || front > rear)
    {
        cout << "Queue is empty\n";
    }
    else
    {
        cout << "Queue elements:\n";

        for (int i = front; i <= rear; i++)
        {
            cout << queueArr[i] << " ";
        }

        cout << endl;
    }
}

int main()
{
    enqueue(10);
    enqueue(20);
    enqueue(30);

    display();

    peek();

    dequeue();

    display();
}

```

```
    return 0;  
}
```

Types of Queue

There are mainly 4 types of queues:

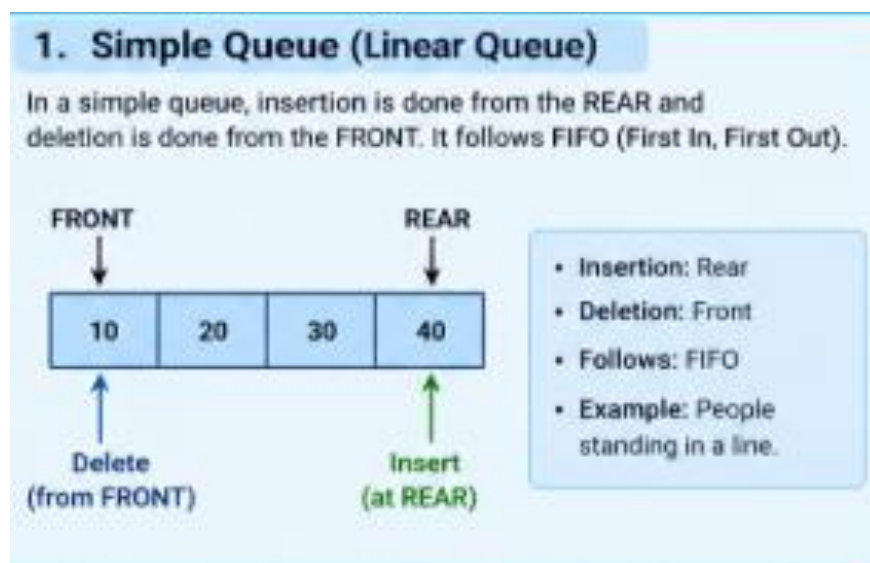
1. Simple Queue (Linear Queue)

In a simple queue, insertion is done from the REAR and deletion is done from the FRONT.

It follows FIFO (First In, First Out).

FRONT → | 10 | 20 | 30 | ← REAR
 ↑ ↑
 Delete Insert

Example: People standing in a line.



2. Circular Queue

In a circular queue, the last position is connected back to the first position.

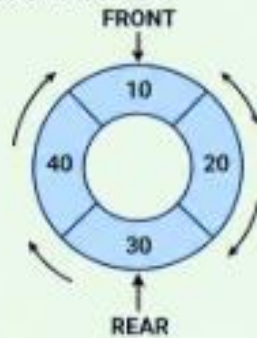
This helps to reuse empty spaces created after deletion.



Example: CPU scheduling, memory management.

2. Circular Queue

In a circular queue, the last position is connected back to the first position. This helps to reuse empty spaces created after deletion.



- Insertion: Rear
- Deletion: Front
- Follows: FIFO
- Last position is connected to first.
- Example: CPU scheduling, memory management.

3. Priority Queue

In a priority queue, each element has a priority. The element with higher priority is processed first, regardless of when it was inserted.

Example:

Element	Priority
10	2
20	1
30	3

If priority 1 is highest, 20 will be processed first.

Example: Hospital emergency system, CPU scheduling.

3. Priority Queue

In a priority queue, each element has a priority. The element with higher priority is processed first, regardless of when it was inserted.

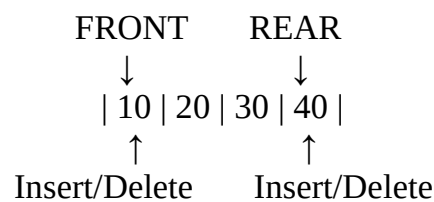
Element	Priority
10	2
20	1
30	3
40	4



- Insertion: Based on priority
- Deletion: Based on priority
- Example: Hospital emergency system, CPU scheduling.

4. Double-Ended Queue (Deque)

A Deque allows insertion and deletion from both FRONT and REAR.



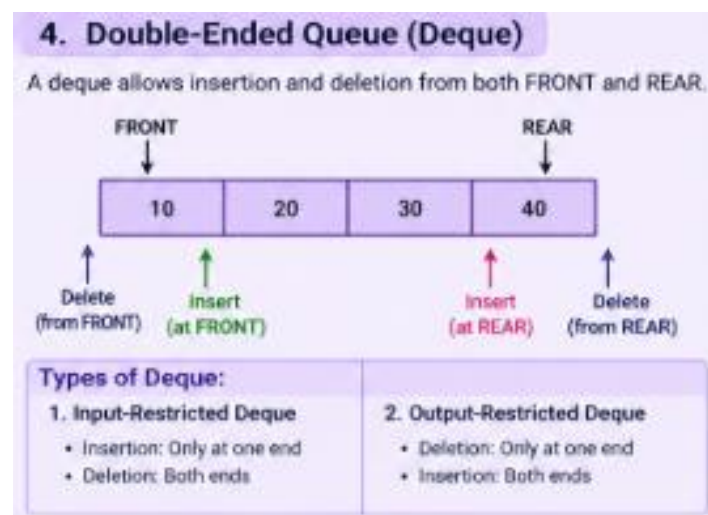
There are two common types:

Input-Restricted Deque:

Insertion is allowed at only one end, but deletion is allowed at both ends.

Output-Restricted Deque:

Deletion is allowed at only one end, but insertion is allowed at both ends.

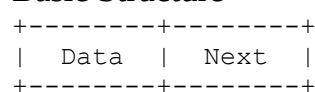


❖ Linked List

A **Linked List** is a linear data structure in which elements are stored in separate units called **nodes**. Each node contains **data** and a **pointer (link)** that stores the address of the next node.

Unlike an array, linked-list nodes are not required to be stored in continuous memory locations.

Basic Structure



Example:

HEAD



[10 | •] → [20 | •] → [30 | NULL]

Types of Linked List

There are mainly four types.

1. Singly Linked List

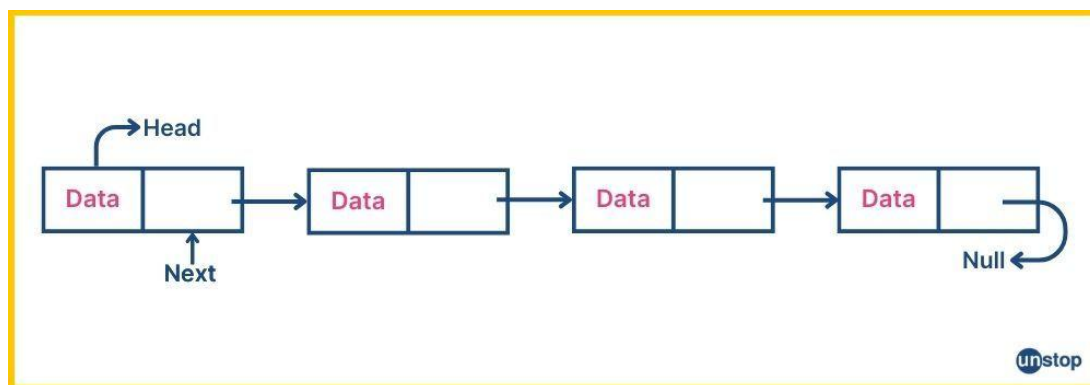
Each node contains data and one next pointer.

HEAD



[10|•] → [20|•] → [30|NULL]

It can be traversed only in the forward direction.



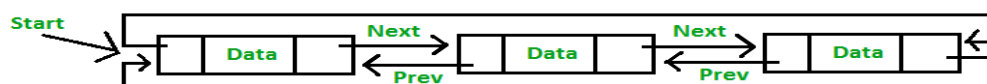
2. Doubly Linked List

Each node contains:

- Previous pointer
- Data
- Next pointer

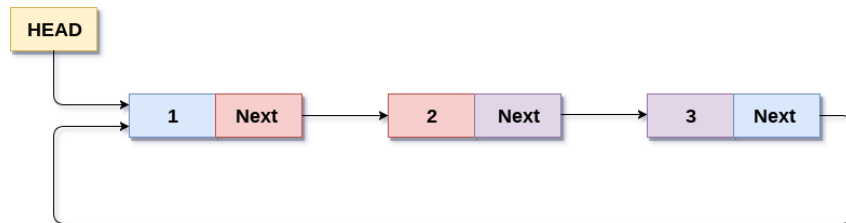
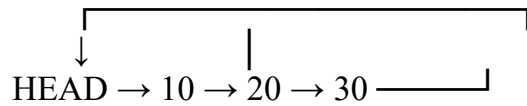
NULL ← [10] ⇌ [20] ⇌ [30] → NULL

It can be traversed in both forward and backward directions.



3. Circular Singly Linked List

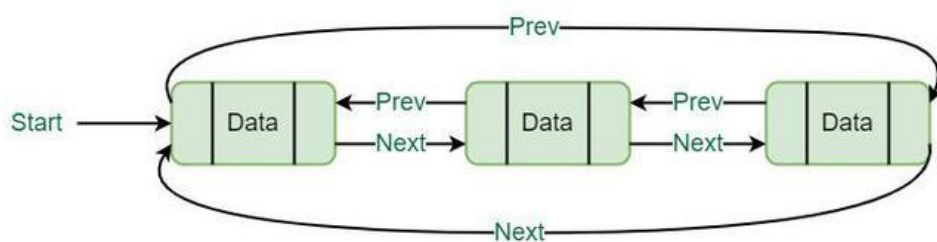
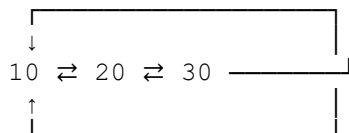
The last node points back to the first node instead of NULL.



Circular Singly Linked List

4. Circular Doubly Linked List

Each node has previous and next pointers, and the last node is connected to the first node.



Basic Operations of Linked List

1. Creation:

Creation is the process of **creating nodes and connecting them** to form a linked list

Example:

HEAD → 10 → 20 → 30 → NULL

2. **Traversal:**

Traversal is the process of **visiting each node one by one** to access or display its data.

Example:

10 → 20 → 30 → NULL

Output: 10 20 30

3. **Insertion:**

Insertion is the process of **adding a new node** to a linked list.

Example:

Before:

10 → 20 → 30

Insert 15:

After:

10 → 15 → 20 → 30

4. **Deletion:**

Deletion is the process of **removing an existing node** from a linked list.

Example:

Before:

10 → 20 → 30

Delete 20:

After:

10 → 30 → NULL

5. **Searching:**

Searching is the process of **finding a specific data element** in a linked list.

Example:

10 → 20 → 30 → NULL

Search = 20

Result = Element Found

6.Updating:

Updating is the process of **changing the data of an existing node** in a linked list.

Example:

Before:

10 → 20 → 30 → NULL

Update 20 to 25

After:

10 → 25 → 30 → NULL

Write a C++ Program to implement LinkedList Operation.

```
#include <iostream>
using namespace std;

class Node {
public:
    int data;
    Node* next;

    Node(int value) {
        data = value;
        next = NULL;
    }
};

// Display the linked list
void display(Node* head) {
    Node* temp = head;

    while (temp != NULL) {
        cout << temp->data << " -> ";
        temp = temp->next;
    }

    cout << "NULL" << endl;
}

// Insert at beginning
void insertBeginning(Node*& head, int value) {
```

```

Node* newNode = new Node(value);

newNode->next = head;
head = newNode;
}

// Insert at end
void insertEnd(Node*& head, int value) {
    Node* newNode = new Node(value);

    if (head == NULL) {
        head = newNode;
        return;
    }

    Node* temp = head;

    while (temp->next != NULL) {
        temp = temp->next;
    }

    temp->next = newNode;
}

// Delete from beginning
void deleteBeginning(Node*& head) {
    if (head == NULL) {
        cout << "List is empty" << endl;
        return;
    }

    Node* temp = head;
    head = head->next;

    delete temp;
}

int main() {

    Node* head = NULL;

    // Insert elements
    insertEnd(head, 10);
    insertEnd(head, 20);
    insertEnd(head, 30);

```

```

cout << "Original Linked List: ";
display(head);

// Insert at beginning
insertBeginning(head, 5);

cout << "After insertion at beginning: ";
display(head);

// Insert at end
insertEnd(head, 40);

cout << "After insertion at end: ";
display(head);

// Delete from beginning
deleteBeginning(head);

cout << "After deletion from beginning: ";
display(head);

return 0;
}

```

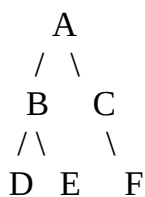
❖ TREE

Definition

A tree is a non-linear data structure in which elements are arranged in a hierarchical structure.

A tree consists of nodes connected by edges.

Simple example



Here:

- A is the **root**
- B, C are children of A
- D, E are children of B

- F is a child of C
- Lines connecting nodes are called **edges**

Important Terminology of Tree

1. Node

An individual element of a tree is called a **node**.

Example:

A

A is a node.

2. Root

The topmost node of a tree is called the **root**.

```

  A
 / \
B   C

```

A is the root.

3. Edge

The connection between two nodes is called an **edge**.

A ----- B

The line between A and B is an edge.

4. Parent

A node that has one or more child nodes is called a **parent**.

```

  A
 / \
B   C

```

A is the parent of B and C.

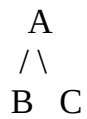
5. Child

A node directly connected below another node is called its **child**.

B and C are children of A.

6. Sibling

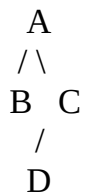
Nodes having the same parent are called **siblings**.



B and C are siblings.

7. Leaf Node

A node that has **no children** is called a leaf node.



B and D are leaf nodes.

8. Degree of Node

The number of children of a node is called its **degree**.



Degree of A = 3.

9. Depth

The number of edges from the root to a particular node is called its **depth**.

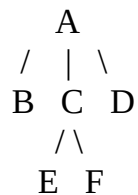
10. Height

The longest path from a node to a leaf node is called its **height**.

Types of Trees

1. General Tree

A tree in which a node can have any number of children.

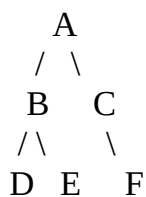


2. Binary Tree

A tree in which each node can have **at most two children**.

The two children are called:

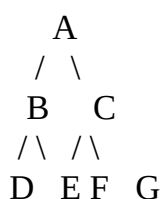
- Left child
- Right child



Each node has maximum two children.

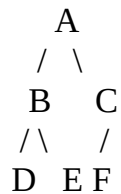
3. Full Binary Tree

A binary tree in which every node has either **0 or 2 children**.



4. Complete Binary Tree

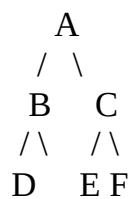
A binary tree in which all levels are completely filled except possibly the last level, and the last level is filled from **left to right**.



5. Perfect Binary Tree

A binary tree in which:

- Every internal node has exactly 2 children.
- All leaf nodes are at the same level.

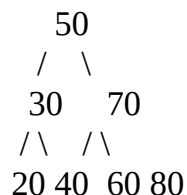


6. Binary Search Tree (BST)

A Binary Search Tree follows this rule:

Left subtree < Root < Right subtree

Example:



Searching becomes easier because values are organized according to their order.

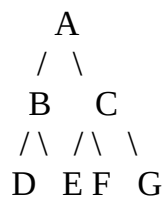
Tree Traversal

Tree traversal means visiting every node of a tree.

The main types are:

1. Preorder
2. Inorder
3. Postorder
4. Level Order

Consider:



1. Preorder

Order:

Root → Left → Right

A → B → D → E → C → F → G

2. Inorder

Order:

Left → Root → Right

D → B → E → A → F → C → G

3. Postorder

Order:

Left → Right → Root

D → E → B → F → G → C → A

4. Level Order

Nodes are visited level by level.

A → B → C → D → E → F → G

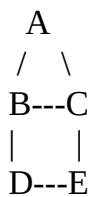
❖ GRAPH

Definition

A **graph** is a non-linear data structure consisting of a collection of:

- **Vertices (Nodes)**
- **Edges (Connections)**

Example:



Here:

- A, B, C, D, E are vertices
- Lines connecting them are edges

A graph does **not necessarily have a root** or hierarchical structure.

Graph Terminology

1. Vertex

A point/node in a graph is called a **vertex**.

Example:

A

A is a vertex.

2. Edge

A connection between two vertices is called an **edge**.

A ----- B

3. Adjacent Vertices

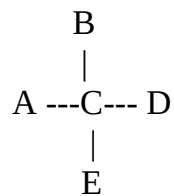
Two vertices connected directly by an edge are called **adjacent vertices**

A ----- B

A and B are adjacent.

4. Degree

The number of edges connected to a vertex is called its **degree**.



Degree of C = 4.

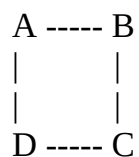
5. Path

A sequence of vertices connected by edges is called a **path**.

A → B → C → D

6. Cycle

A path that starts and ends at the same vertex is called a **cycle**.



A → B → C → D → A is a cycle.

7. Types of Graphs

1. Undirected Graph

Edges do not have a direction.

A ----- B

You can travel from A to B and B to A.

2. Directed Graph

Edges have a specific direction.

A -----> B

You can travel from A to B, but not necessarily from B to A.

3. Weighted Graph

Edges have values or weights.

```
A --10-- B
|         |
5         8
|         |
C --12-- D
```

The numbers represent distance, cost, time, etc.

4. Unweighted Graph

Edges do not have weights.

```
A ----- B
|         |
C ----- D
```

5. Connected Graph

A graph is connected if there is a path between every pair of vertices.

```
A ----- B
|         |
C ----- D
```

All vertices are connected.

6. Disconnected Graph

A graph containing separate components.

A ----- B C ----- D

There is no connection between the two groups.

Graph Representation

Graphs are commonly represented using:

1. Adjacency Matrix

An adjacency matrix uses a **2D array**.

Example:

	A	B	C
A	0	1	1
B	1	0	1
C	1	1	0

1 means an edge exists.

0 means no edge exists.

2. Adjacency List

Each vertex stores a list of its connected vertices.

Example:

A → B → C
B → A → C
C → A → B

Adjacency lists are useful when the graph has fewer edges compared with the possible number of edges.

Graph Traversal

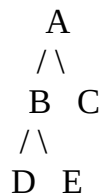
There are two major graph traversal techniques:

1. BFS – Breadth First Search

BFS visits nodes **level by level**.

It generally uses a **queue**.

Example:



BFS:

$A \rightarrow B \rightarrow C \rightarrow D \rightarrow E$

Algorithm

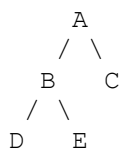
1. Start from a vertex.
2. Mark it as visited.
3. Insert it into a queue.
4. Remove a vertex from the queue.
5. Visit its unvisited adjacent vertices.
6. Add them to the queue.
7. Repeat until the queue becomes empty.

2. DFS – Depth First Search

DFS goes as deep as possible before coming back.

It generally uses a **stack** or recursion.

For the same graph:



One possible DFS:

$A \rightarrow B \rightarrow D \rightarrow E \rightarrow C$

Algorithm

1. Start from a vertex.
2. Mark it as visited.
3. Visit an unvisited adjacent vertex.
4. Continue going deeper.
5. Backtrack when no unvisited vertex is available.
6. Repeat until all required vertices are visited.